

Project 1: Handwritten Digit Recognition using ANN

Step 1: Import Libraries

What did I do? Imported TensorFlow, Matplotlib, NumPy, Sequential, Flatten and Dense.

Why? Build, train and visualize the ANN.

Output: Libraries loaded successfully.

Step 2: Load Dataset

What did I do? Loaded MNIST dataset into X_train, y_train, X_test and y_test.

Why? Training data teaches the model; testing data evaluates it.

Output: 60000 train images, 10000 test images.

Step 3: Explore Dataset

What did I do? Checked dataset shapes.

Why? Verified dataset dimensions.

Output: Train:(60000,28,28), Test:(10000,28,28).

Step 4: Visualize Images

What did I do? Displayed the first handwritten digit.

Why? Verified image-label correspondence.

Output: First digit displayed.

Step 5: Check Pixel Values

What did I do? Printed the image matrix.

Why? Images are stored as numerical pixel values.

Output: 28×28 matrix with values 0-255.

Step 6: Normalize

What did I do? Divided pixel values by 255.

Why? Improves convergence and training stability.

Output: Values between 0 and 1.

Step 7: Build ANN

What did I do? Created Sequential model with Flatten, Dense(128,ReLU), Dense(10,Softmax).

Why? Construct the neural network.

Output: ANN architecture created.

Step 8: Model Summary

What did I do? Viewed layers and parameters.

Why? Understand architecture and trainable parameters.

Output: Layer details shown.

Step 9: Compile

What did I do? Configured Adam optimizer, sparse categorical crossentropy loss and accuracy metric.

Why? Prepare model for training.

Output: Model ready for training.

Step 10: Train

What did I do? Called `model.fit` with `epochs=10`, `batch_size=32`, `validation_split=0.2`.

Why? Model learns patterns by updating weights.

Output: Training/validation loss and accuracy displayed.

Step 11: Evaluate

What did I do? Evaluated on test set.

Why? Measure generalization.

Output: Test loss and accuracy returned.

Step 12: Print Results

What did I do? Printed test metrics.

Why? View final performance.

Output: Accuracy and loss displayed.

Step 13: Predict

What did I do? Generated predictions using `model.predict`.

Why? Predict unseen images.

Output: Probability vector for each image.

Step 14: Predicted Class

What did I do? Used `argmax` to select highest probability.

Why? Convert probabilities to class label.

Output: Predicted digit obtained.

Step 15: Compare

What did I do? Compared predicted and actual labels.

Why? Verify correctness.

Output: Prediction vs actual shown.

Step 16: Visualize Prediction

What did I do? Displayed image with predicted and actual labels.

Why? Visual confirmation.

Output: Prediction visualization.

Step 17: Save Model

What did I do? Saved model as `mnist_ann_model.keras`.

Why? Reuse without retraining.

Output: Model file created.